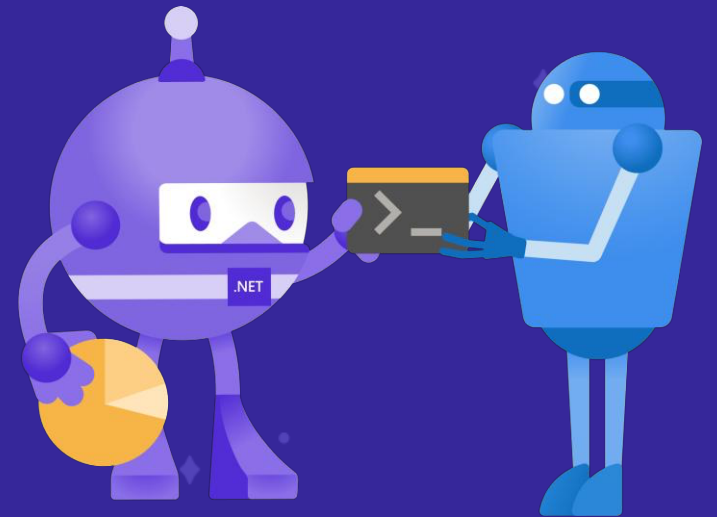


Copilot SDK로 나만의 AI Agent 빌드하기

멀티플랫폼 SDK로 GitHub Copilot Agent를
앱과 서비스에 통합해 나만의 AI Agent를 빌드해보세요

디모이 (dimohy@slogs.dev)



왜 지금 AI Agent인가

from answers to actions

- 관심사는 답변형 AI에서 행동형 Agent로 이동
- 중요한 것은 말 잘하는 AI보다 실제 업무를 수행하는 능력
- 개발자는 Agent를 사용하는 단계에서 제품에 내장하는 단계로 이동 중
- Copilot SDK는 그 전환을 가장 빠르게 시작할 수 있는 방법

답변형 **AI**에서 행동형 **Agent**로 중심축이 이동 중

Copilot SDK란?

- GitHub Copilot Agent runtime을 프로그래밍 방식으로 사용하는 SDK
- TypeScript, Python, Go, .NET 지원
- Copilot CLI와 JSON-RPC로 통신하며 세션과 이벤트를 관리
- 단순 LLM 호출 라이브러리가 아니라 Agent runtime SDK
- 현재는 Technical Preview 단계

runtime SDK

JSON-RPC

.NET · TS · Py · Go

Built on GitHub Copilot CLI

Reuse

CLI가 이미 가진 Agent runtime을 그대로 재활용

Modes

로컬 CLI 또는 서버 모드에 맞춰 통합 가능

Control

세션 · 이벤트 · 톨을 코드로 오케스트레이션

핵심 가치

새로운 모델 API가 아니라 검증된 Copilot Agent runtime을 제품 안으로 가져오는 것

Copilot CLI vs Copilot SDK

interactive vs embedded

Copilot CLI

1. 터미널에서 바로 쓰는 ready-to-use assistant
2. 개발자 개인 생산성 향상에 최적화
3. 탐색, 디버깅, 빠른 수정에 강함

Copilot SDK

1. 앱과 서비스 안에 Agent 기능 임베드
2. 세션, 이벤트, 툴, 권한을 코드로 제어
3. 제품 기능으로 AI workflow를 제공할 때 적합

SDK가 가져오는 능력

Context

멀티 파일 컨텍스트와 코드 수정

Tools

Shell · 파일 · 웹 도구 실행

MCP

외부 도구를 표준 방식으로 연결

Hooks

Lifecycle hooks와 다양한 운영 모드

핵심 메시지

CLI의 힘을 내 앱 안으로 가져오는 것

앱 안에서 Agent는 어떻게 동작할까



Files

Web

Git

MCP

앱이 SDK를 통해 Agent runtime과 연결되는 구조

인증 패턴

GitHub Identity / OAuth

1. GitHub 계정 기반으로 Copilot 구독 활용
2. 사용자별 인증이 필요한 앱에 자연스러움
3. 웹앱이나 팀용 도구에서 직관적

BYOK

1. 자체 API Key를 사용해 모델 공급자 연결
2. 중앙 과금과 운영 통제가 쉬움
3. 사내 서비스나 단일 서비스 계정 시나리오에 적합

선택 기준: 누가 비용을 내고 누구 권한으로 **Agent**를 실행하는가

통합 모델: 로컬 vs 원격

Local Integration

1. SDK와 CLI가 같은 머신에서 동작
2. 가장 단순한 시작 방식
3. 단일 사용자 도구나 데스크톱 앱에 적합

prototype friendly

Remote Integration

1. SDK와 CLI 서버가 다른 머신에서 동작
2. 서버 모드 CLI와 연결해 확장 가능
3. 웹 서비스나 중앙 집중형 아키텍처에 적합

service ready

패키징 · CLI · 토큰 운영

SDK 패키징

- NuGet: GitHub.Copilot.SDK 참조
 - 배포: dotnet publish / self-contained / container
- 결과물에 SDK 어셈블리 포함

CLI 런타임

- 사전 설치된 Copilot CLI 연결 가능
 - 또는 호환 CLI bundle/install 흐름 사용
- 원격 환경은 CLI server mode 연결

인증 / 토큰 운영

- 기본은 GH_TOKEN 우선, 없으면 로그인 사용자 문맥 사용
 - 헤드리스 환경은 GH_TOKEN / GITHUB_TOKEN 을 secret로 주입
 - PAT는 Developer settings에서 Fine-grained token 우선 발급

코드 흐름: **CopilotClient** 생성 → **StartAsync()** → **CreateSessionAsync(...)**

운영 원칙: 토큰은 브라우저가 아니라 서버/**CI secret store**에만 저장

내부 앱과 웹앱에 붙이는 방식



“채팅 **UI**”보다 “백엔드 워크플로우 통합” 관점이 중요

Build an Agent in .NET

가장 작은 콘솔 앱부터 시작해서 **Streaming, Tool**, 확장 포인트까지 빠르게 살펴봅니다.

가장 작은 시작

```
using GitHub.Copilot.SDK;

await using var client = new CopilotClient();
await client.StartAsync();
await using var session =
    await client.CreateSessionAsync(new() { Model = "gpt-4.1" });
var response = await session.SendAndWaitAsync(
    new() { Prompt = "What is 2 + 2?" });
Console.WriteLine(response?.Data.Content);
```

Streaming Events

```
await using var session = await client.CreateSessionAsync(new()  
{  
    Model = "gpt-4.1",  
    Streaming = true  
});  
  
session.On(ev =>  
{  
    if (ev is AssistantMessageDeltaEvent delta)  
        Console.Write(delta.Data.DeltaContent);  
});
```

Custom Tool 연결

```
var lookupIssue = AIFunctionFactory.Create(  
    ([Description("이슈 번호")] string issueId) =>  
        new { issueId, status = "Open" },  
    "lookup_issue",  
    "이슈 상태를 조회합니다.");
```

```
await using var session = await client.CreateSessionAsync(new()  
{  
    Model = "gpt-4.1",  
    Tools = [lookupIssue]  
});
```

데모에서 보여줄 것

what to watch

- 연동한 GitHub Copilot SDK의 설정 상태 확인
- JAVA/XML + 테이블 정의서 기반 Flow JSON 생성과 Diagram 시각화
- Flow JSON을 이용한 업무 친화적인 AI 어시스턴트 응답
- 설정 → 생성 → 활용 흐름이 한 번에 이어지는 데모

핵심은 단일 모델 호출이 아니라 메타정보 생성부터 업무 응답까지 이어지는 흐름

LIVE DEMO

- **GitHub Copilot SDK**를 이용해서 소스코드와 **SQL** 쿼리를 이용 **Flow** 메타정보를 생성해서 업무 친화적인 **AI** 어시스턴스 제공

<https://metagraphai.slogs.dev>



스크린샷 1. 연동한 GitHub Copilot SDK의 설정

설정

GitHub Copilot SDK 실행 준비 상태와 실제 세션 생성 가능 여부를 확인합니다.

GH 토큰 설정

저장된 토큰은 현재 사용자 계정 기준으로 보호 저장되며, GitHub Copilot SDK 실행 시 'GH_TOKEN'으로 주입됩니다.

GH_TOKEN

토큰 저장토큰 삭제

저장된 GH 토큰이 있습니다.

GitHub Copilot SDK 상태 점검

확인 버튼을 누르면 SDK 시작, 모델 목록 조회, 세션 생성 가능 여부를 순서대로 검사합니다.

SDK 동작 확인

SDK 패키지: Y
SDK 시작: Y
모델 조회: Y
세션 생성: Y
인증 상태: Y
GH 토큰 설정: Y

실행 모드: SDK 직접 런타임(copilot CLI:
/home/dimohy/MetagraphAi/current/runtimes/linux-x64/native/copilot)
SDK 버전: 0.1.29.0
확인 시각: 2026-03-11 19:33:02

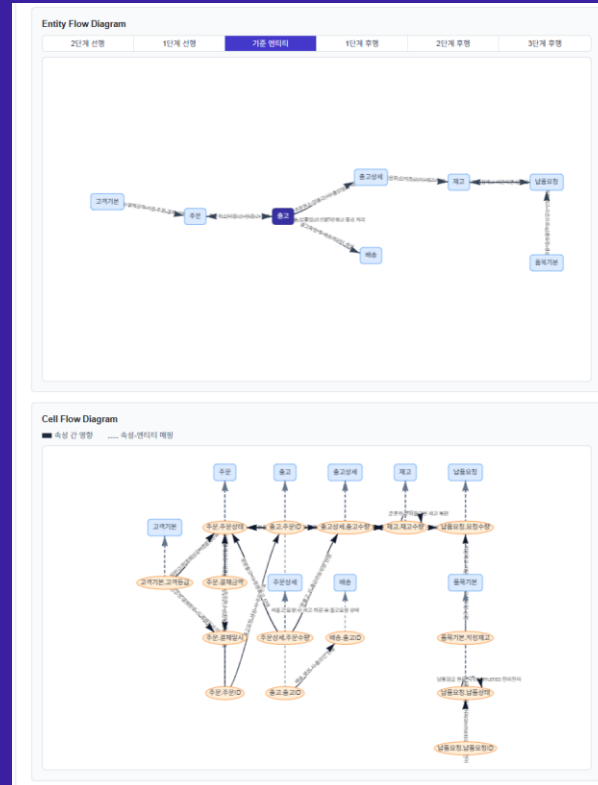
SDK 정상: CLI 시작 OK (State=Connected) | Ping OK (ts=1773225181677) | 인증 OK | 세션 생성 OK (model=claude-haiku-4.5, id=14baee62...)

포인트

- GH_TOKEN 저장
- SDK 시작 확인
- 모델 조회 / 세션 생성 가능

데모 시작 전에 **SDK**가 실제로 동작 가능한 상태인지 먼저 검증

스크린샷 2. JAVA/XML + 테이블 정의서 기반 Flow JSON 생성과 Diagram 시각화



포인트

- JAVA/XML 입력
- 테이블 정의서 업로드
- Flow JSON 생성 + Diagram 시각화

소스 기반/데이터 기반 근거를 **Flow** 메타정보로 변환하고 시각화까지 연결

스크린샷 3. Flow JSON을 이용한 업무 친화적인 AI 어시스턴스

대화

응답 중지 대화 초기화

도메인 선택

test260311

목록 새로고침

사용자

모든 발송 대기 목록을 조회해줘

AI- Copilot Agent SDK (claude-haiku-4.5) (JSON-Pipeline)

확인 결과, 현재 시스템에서 배송 대기 중인 주문은 다음과 같습니다.

[배송 대기 목록]

| 주문번호 | 고객명 | 주문일시 | 주문금액 | 현재상태 | 결제완료 |

|-----|-----|-----|-----|-----|-----|

| 2025000001 | 김소영 | 11/20/2025 6:01 PM | 1,550,000원 | 출고요청 | 완료 |

| 2025000003 | 김미숙 | 1/8/2025 11:30 AM | 420,000원 | 출고요청 | 완료 |

| 2025000008 | 김일한 | 1/20/2025 8:50 AM | 75,000원 | 출고요청 | 완료 |

| 2025000013 | 조서윤 | 1/30/2025 9:20 AM | 170,000원 | 출고요청 | 완료 |

| 2025000017 | 윤하정 | 2/7/2025 2:40 PM | 1,100,000원 | 출고요청 | 완료 |

질문 입력

예: 이름이 뭐야?

보내기

포인트

- 선택한 JSON 근거 사용
- 업무 친화적 질의응답
- Flow 근거를 함께 제시

생성된 **Flow JSON**이 실제 업무 맥락의 **AI** 어시스턴트 응답 근거로 이어짐

어디에 붙이면 좋을까

Internal Ops

사내 운영 도구
운영/지원 팀용 Agent

Dev Portal

문서 · 이슈 · 배포 Assistant
개발 생산성 포털

Product Feature

웹앱 내 Agent Assistant
관리자 도구 자동화

Best Fit

처음부터 만능 챗봇보다 좁고 깊은 workflow에 특화된 Agent가 성공
확률이 높음

여기서부터 진짜 확장

Hooks

툴 실행 전후 제어, 프롬프트 수정, 오류 처리

MCP Servers

외부 도구를 표준 방식으로 연결

Custom Agents

특정 역할의 전문 Agent 정의

Skills

재사용 가능한 지식과 프롬프트 모듈

BYOK

GitHub 외 모델 공급자 사용 가능 — 내부 서비스와 비용 모델을 유연하게 선택

오늘 기억할 3가지

1. 내 앱 안에 **Agent** 넣기

Copilot SDK는 Agent runtime을 제품 기능으로 바꾸는 방법

2. 시작은 단순하게

Client → Session → Prompt 만으로도 첫 경험을 만들 수 있음

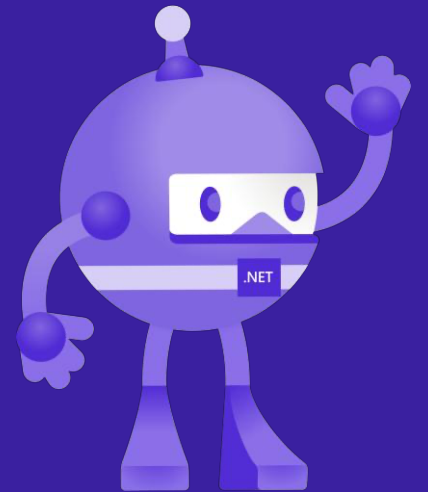
3. 확장이 진짜 힘

Tool / Hook / MCP / Agent 조합으로 도메인에 맞는 Agent를 구축

.NET에서도 충분히 자연스럽고 실전적

Thank you!

들어주셔서 감사합니다.



부록

GitHub Copilot SDK / VS Code 커스터마이징 한눈에 보기

무엇을 담았나

- .NET/C# Custom Tool 연결
- MCP / Skills / `AGENTS.md`
- Ask / Plan / Agent 선택
- `SystemMessageMode.Replace`

출처: GitHub Copilot SDK repo · VS Code 공식 Copilot 문서

문서 성격

- 발표용 빠른 참조
- 공식 문서 기반 요약
- 실습 중 헛갈리는 포인트 중심 정리

가장 먼저 봐야 할 페이지

① .NET Custom Tool 입문

[docs/getting-started.md#step-4-add-a-custom-tool](#)

- 가장 짧고 이해가 빠름
- `AIFunctionFactory.Create( ... )` 예제 포함
- `SessionConfig.Tools` 연결 흐름 확인 가능

② .NET SDK 상세 레퍼런스

[dotnet/README.md](#) → **Advanced Usage > Tools**

- 도구 동작 방식 설명
- 반환값/메타데이터 개념
- 내장 도구 override 방법까지 포함

발표에서 “.NET으로 Custom Tool 어떻게 붙이나요?”가 나오면 먼저 ①, 심화 질문은 ②로 연결하면 된다.

.NET Custom Tool 연결 핵심

개념

- 내 C# 함수를 Copilot이 호출 가능한 도구로 노출
- 보통 `Microsoft.Extensions.AI` 의 `AIFunctionFactory.Create` 사용
- 세션 생성 시 `SessionConfig.Tools` 에 등록

언제 쓰나

- 내 앱 로직을 직접 호출시키고 싶을 때
- 사내 API 래퍼 / 계산 함수 / 조회 함수 연결
- MCP 서버까지는 과한 작은 기능 연결

```
var getWeather = AIFunctionFactory.Create(  
    [Description("The city name")] string city) =>  
    new { city, temperature = "72°F", condition = "sunny" },  
    "get_weather",  
    "Get the current weather for a city"  
);  
  
await using var session = await client.CreateSessionAsync(new SessionConfig  
{  
    Model = "gpt-4.1",  
    Tools = [getWeather],  
});
```

MCP는 무엇이 다른가

Custom Tool

내 프로세스 안의 함수

- 간단
- 직접 제어
- 앱 내부 기능 연결에 적합

MCP Server

외부 프로세스/서버의 도구 집합

- 재사용성 높음
- GitHub/DB/브라우저형에 적합
- 서버 단위로 불임

실무 감각

함수 몇 개면 Tool, 툴 플랫폼이면 MCP

- 작으면 Tool
- 외부 시스템이면 MCP
- 팀 공용이면 MCP 쪽이 유리

```
await using var session = await client.CreateSessionAsync(new SessionConfig
{
    Model = "gpt-5",
    McpServers = new Dictionary<string, object>
    {
        ["my-local-server"] = new McpLocalServerConfig
        {
            Type = "local",
            Command = "node",
            Args = new List<string> { "./mcp-server.js" },
            Tools = new List<string> { "*" },
        },
    },
});
```

Skills는 지침 모듈이다

정체

- `SKILL.md` 를 가진 디렉터리
- 도구가 아니라 **행동 규칙·체크리스트·도메인 지식**
- 세션 컨텍스트에 주입되는 재사용 모듈

언제 좋나

- 보안 리뷰 규칙
- 문서 작성 기준
- 테스트 전략 / 운영 가이드
- MCP·Custom Agent와 조합

```
skills/  
├── code-review/  
│   └── SKILL.md  
└── documentation/  
    └── SKILL.md
```

```
SkillDirectories = new List<string> { "./skills/code-review", "./skills/documentation" }
```

정책 파일 vs 커스텀 에이전트

AGENTS.md

- 항상 적용되는 워크스페이스 정책
- 프로젝트 규칙 / 아키텍처 / 빌드 지침
- 루트 또는 서브폴더별 계층 적용 가능

성격: 기본 헌법 📖

.agent.md

- 선택 가능한 역할 기반 에이전트
- 도구 제한 / 모델 지정 / handoff 가능
- 플래너, 리뷰어, 구현자 같은 역할 분리

성격: 직무별 전문가 카드 🧑‍💻

AGENTS.md = 정책

.agent.md = 역할

둘은 이름만 비슷하고 용도는 다름

Ask / Plan / Agent 선택법

Ask

- 질문/설명 중심
- 파일 변경 없음
- 개념 확인, 비교 설명, 빠른 Q&A

Plan

- 구현 전 설계
- 순서, 범위, 리스크 정리
- 바로 코딩 전에 유용

Agent

- 수정·실행·도구 호출
- 멀티파일 작업
- 실제 구현 담당

VS Code에서 어디서 고르나?

Chat View의 **agent picker** 에서 선택한다. 여기에 실행 위치(Local / Background / Cloud)와 모델 선택도 함께 조합한다.

SDK 안에서의 "Ask" 흐름

UI의 Ask 모드와 다름

- VS Code Ask = 질문 전용 채팅 모드
- SDK Ask-like = 에이전트가 사용자에게 되묻는 흐름

핵심 API

- `OnUserInputRequest`
- 에이전트가 추가 질문이 필요할 때 호출
- 콘솔/앱 UI에 연결 가능

```
OnUserInputRequest = async (request, invocation) =>
{
    Console.WriteLine($"Agent asks: {request.Question}");
    return new UserInputResponse
    {
        Answer = "User's answer here",
        WasFreeform = true
    };
}
```

시스템 프롬프트를 바꾸는 법

Append

- 기본 시스템 프롬프트에 추가
- 가드레일 유지
- 대부분 이쪽이 안전

Replace

- 기본 시스템 프롬프트를 대체
- 강력하지만 책임도 큼
- 안전/형식/도구 정책을 직접 써야 함

```
SystemMessage = new SystemMessageConfig
{
    Mode = SystemMessageMode.Replace,
    Content = "You are a helpful assistant."
}
```

권장 순서: **Append 먼저** → 정말 필요할 때만 **Replace**

무엇을 언제 쓰면 좋은가

목적	선택
내 C# 함수 하나를 도구로 연결	Custom Tool
외부 도구 서버 연결	MCP
행동 규칙/체크리스트 재사용	Skill
저장소 전역 정책	copilot-instructions.md / AGENTS.md
역할별 전문 에이전트	.agent.md / CustomAgents
기본 시스템 프롬프트 확장	Append
시스템 프롬프트 완전 교체	Replace

추천 조합

- 실무형 도우미 앱 → **Custom Tool + Append**
- 외부 시스템 연동형 → **MCP + Custom Agent**
- 팀 규칙 강제형 → **AGENTS.md + .instructions.md**
- 발표 데모형 → **Custom Tool + MCP + Ask/Agent 비교**

발표용 한 줄 정리

핵심 문장

- .NET Custom Tool 입문은 `Getting Started` Step 4
- 심화는 `dotnet/README.md` 의 **Advanced Usage > Tools**
- 외부 도구 연결은 **MCP**
- 재사용 규칙 묶음은 **Skills**

기억 포인트

- `AGENTS.md` = 정책
- `.agent.md` = 역할
- VS Code는 **Ask / Plan / Agent** picker 제공
- 시스템 프롬프트 전체 교체는 `SystemMessageMode.Replace`

이제 질문이 들어와도 덜 헛갈리고, 데모도 덜 떨린다. 문서가 대신 기억해준다 😎